Velocity Obstacle Based Strategy for Multi-agent Collision Avoidance of Unmanned Aerial Vehicles

Alexandre Lombard
CIAD UMR 7533, Univ. Bourgogne Franche-Comté, UTBM
Belfort, France
alexandre.lombard@utbm.fr

Lilian Durand
FISE Informatique, Univ. Bourgogne Franche-Comté, UTBM
Belfort, France
lilian.durand@utbm.fr

Stéphane Galland
CIAD UMR 7533, Univ. Bourgogne Franche-Comté, UTBM
Belfort, France
stephane.galland@utbm.fr

***Abstract*—Unmanned Aerials Vehicles (UAVs), most known as drones, recently gained a lot of interest in several domains of civil applications. With the number of operating drones expected to quickly rise in the next few years comes the need to develop efficient collision avoidance solutions. To ensure the applicability of the collision avoidance solutions, given the limitations of UAVs, they must rely on a reliable source of information, and must be computationally efficient. This paper aims at providing an implementation of the Velocity Obstacle collision avoidance method, based on the information transmitted by the other UAVs. Multi-agent based simulations using the simulation software AirSim and the multi-agent platform SARL are proposed to assess the validity of the proposed solution.**

***Index Terms*—unmanned aerial vehicles, collision avoidance, multi-agent systems**

I. Introduction

Unmanned Aerials Vehicles (UAVs) are expected to become numerous to fly in our sky in the next few years, as they will enable, among others, in-the-air services (delivery of goods or food), aerial transport of passengers, in-the-air infrastructure (mobile WiFi or 4G/5G antennas) [1].

To enable all these expected benefits of the UAVs, a lot of research is still on-going on their autonomy, as automated decision making is one of the most difficult problems leading to autonomy. Indeed, the systematic literature review conducted by Mualla et al. [2] highlights that nowadays two of the most important research directions are the model design of fully autonomous UAVs and the verification and validation of the UAVs behaviors. Regarding the design of the fully autonomous UAVs, one of the major key points is the definition of a valid collision avoidance solution.

A UAV is considered as autonomous if no external human pilot is involved for the control of the drone's trajectory. This autonomy leads to the need to define a strategy that will prevent the UAVs from hitting obstacles, both statics (trees, buildings, *etc*.) and mobiles (notably the other drones, birds, *etc*.)

In a continuous environment [3], defining a strategy for avoiding the collision with a static obstacle is an easy task, but it gets harder with mobile obstacles, especially when these mobile obstacles are numerous and they are simultaneously trying to avoid the collisions with all the surrounding potential obstacles. The extension of the algorithms for collision avoidance with static obstacles to mobile obstacles, especially in a situation implying an important number of UAVs in the limited same space, is leading to instability in their behaviors called "jitter" that can be defined as small fluctuations or oscillations on the motion directions.

There exists many ways to perform collision avoidance algorithms for a swarm of autonomous robots, especially solutions that are based on multiagent systems [4]. The most common solutions follow one of the following approaches:

- *Geometric*: geometric approaches determine potential collisions by simulating the trajectory of the UAV and of the obstacles, and compute a path avoiding the collision [5];
- *Bearing angle*: it uses a visual sensor and its ability to accurately return the relative angle of the obstacles toward the UAV. By keeping the obstacles' images in a "safe" position in the sensor field of view, the UAV can prevent the collisions [6];
- *Potential fields*: it is inspired by the concepts of repulsive and attractive electrical force-fields; repulsion forces from obstacles are used to avoid collision [7, 8];
- *Social force models*: similar to the potential fields approach, it is inspired by the social force models where agents are influenced by a set of forces attracting them to their target while repulsing them from the obstacles [9, 10];
- *Velocity Obstacle*: in this approach, each UAV has to know the position, velocity and radius of the obstacles [11, 12, 13, 14]; the collision is avoided by computing a cone representing the set of all the velocities of the UAV which would result in a collision with the obstacles.

These strategies are commonly used for collision avoidance of terrestrial robots or for UAVs, which evolves on a 2D plane. In the case of the UAVs, some other elements should be considered: *(i)* the displacements are in 3D; it adds a dimension where the UAV will have to look for obstacles, but also introduce new possibilities to avoid the collisions, and *(ii)* UAVs could communicate their own data, which provides a reliable source of information for the computation of the collision avoidance algorithm (including positions and velocities of the other drones), that is then not solely based

978-1-7281-1062-2/20/$31.00 2020

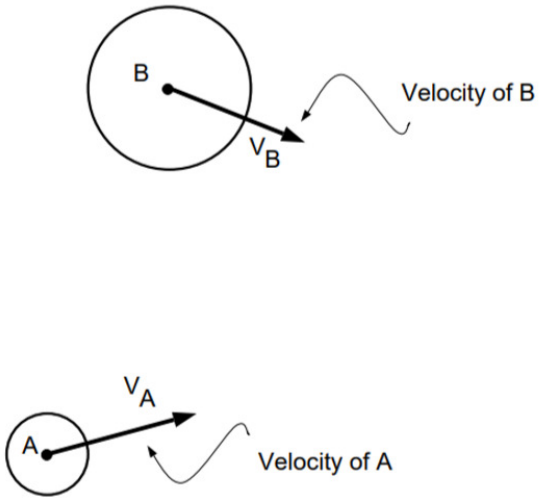


Fig. 1. Planar projection of the representation of two UAVs A and B

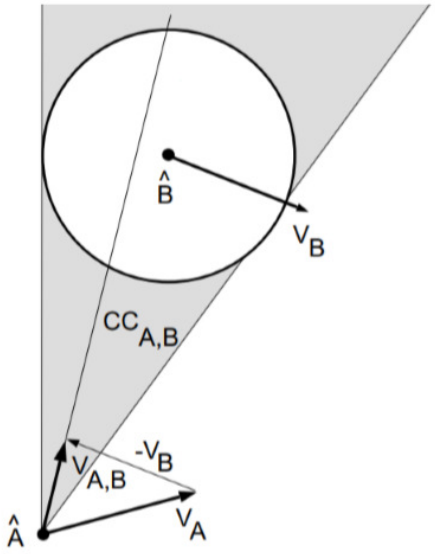


Fig. 2. Collision Cone from UAV A to UAV B

on the UAV's sensors.

Given these elements into consideration, the Velocity Obstacle approach has proven its ability to avoid collision [11, 12, 13, 14]; but it needs to be adapted to work in a 3D motion space. This adaptation enables new ways to manage the collision avoidance maneuver, such as Jenie et al. [13] who propose a solution based on avoidance planes. The present paper proposes another adaptation of the Velocity Obstacle approach in Section II with a collision avoidance maneuver based on a search algorithm of a valid velocity in a 3D space. An improvement is also proposed with a strategy for the dynamic adaptation of the velocity in Section II-C Finally, the simulation methodology and the simulation platform that are used to validate the solution are detailed in the Section III, as well as the simulation results.

II. Velocity Obstacle and Avoidance Maneuver

A. Velocity Obstacle

The *Velocity Obstacle* (VO) strategy introduced by Fiorini and Shiller [11] is based on *Collision Cones* (CC), which will represent the set of velocities that could lead to a collision.

In the case of UAVs, a drone i is described by its position $\hat{i}$ (its geometric center), its velocity v_i (a vector) and its radius r_i representing the spherical bounding box of the drone, centered on its position. It will be considered that if the distance between a drone i and a drone j is below $r_i + r_j$, then there is a collision between the two drones.

To determine the CC from UAV A to UAV B, B must be mapped into the *Configuration Space* of A by representing A by $\hat{A}$ and B by $\hat{B}$. $\hat{A}$ is always located at the origin point of the Configuration Space, and $\hat{B} = B - A$. To represent the set of velocities that generate a collision, r_b is enlarged by r_a (Fig. 2). The CC from A to B is the cone with its apex at $\hat{A}$ and those border lines are tangent to the sphere centered on $\hat{B}$ and with the radius $r_b + r_a$. It is illustrated by the greyed area on Fig. 2. If the velocity vector of A is lying inside the CC, then a collision will eventually occur with B if B is supposed immobile.

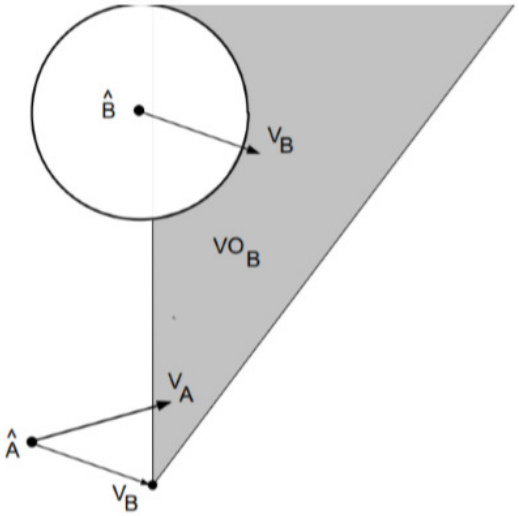


Fig. 3. Velocity Obstacle from UAV A to UAV B

The idea of the VO is to adapt CC to mobile obstacles. In order to do it, the CC has to be translated by the velocity of B $\vec{V_B}$ to determine future collisions (Fig. 3). As the UAV will frequently compute their desired velocity according to a sampling time τ, the effective translation transforming the CC into the VO will be equal to $\tau \times \vec{V_B}$.

VO, as CC, works by pair of UAV/Obstacle. Therefore, one VO has to be computed per obstacle. The global VO can be

computed as the union of all the computed VOs. To compute the VO of another drone, it can be assumed that the UAV can get the other drones' current positions and velocities either relying on wireless transmission of this data, or by using physical sensors (including lidars and cameras). Yet, only the use of wireless transmission allows the perception of drones which are out of sight. For the other obstacles (walls, trees, birds, *etc*.), the UAVs will have to rely on their other sensors.

B. 3D Avoidance Maneuver

Once the set of velocities leading to collision (*i.e*. the VO) is defined, the UAV has to perform an avoidance maneuver in order to move toward its target while avoiding the obstacles. For this, it will have to choose a safe direction (out of the VO) close to the target direction. For this purpose, the Algorithm 1 is applied.

Algorithm 1 Avoidance Maneuver Algorithm

Input:
$vo \subset \mathbb{R}^3$: Set of VO
$\vec{t} \in \mathbb{R}^3$: 3D vector, target of the UAV
Output: $\vec{s_d} \in \mathbb{R}^3$: 3D Vector, safe direction
$\vec{n} \longleftarrow \vec{t} \times \vec{k}$
$\theta \longleftarrow 0$
$\alpha \longleftarrow \alpha_0$
$s \longleftarrow t$
$valid \longleftarrow false$
while $\theta < \frac{\pi}{2}$ **and** $\neg valid$ **do**
  while $\alpha < \alpha_0 + \pi$ **and** $\neg valid$ **do**
    $valid \longleftarrow true$
    $\vec{s_d} \longleftarrow rotateAroundAxis(\vec{t}, \vec{n}, \theta)$
    $\vec{s_d} \longleftarrow rotateAroundAxis(\vec{s}, \vec{t}, \alpha)$
    for all $v \in vo$ **do**
      if $\vec{s_d} \in v$ **then**
        $valid \longleftarrow false$
      end if
    end for
    $\alpha \longleftarrow \alpha + \Delta\alpha$
  end while
  $\theta \longleftarrow \theta + \Delta\theta$
end while

In this algorithm, $\vec{k}$ is the "up" vector, the function $rotateAroundAxis(\vec{v} : \mathbb{R}^3, \vec{a} : \mathbb{R}^3, \theta : \mathbb{R}$ returns the rotation of angle θ of the vector $\vec{v}$ around the axis $\vec{a}$.

The principle of the algorithm is that the drone initializes the *safe direction* s with the direction to its target. As long as the *safe direction* is inside a VO, rotations will be applied to the direction of the vector until a safe vector, *i.e*. a vector out of the VOs is found. This approach aims at limiting the impact of the collision avoidance maneuver by finding a *safe direction* close to the *target direction* as much as possible.

The angle α_0 is used to initialize the second rotation of angle α. It is defined by Equation 1.

$$\alpha_0 = \operatorname{atan2}(y(\vec{t}), x(\vec{t})) + \arcsin z(\vec{t}) \tag{1}$$

As the algorithm stops as soon as a safe direction is found, the basic idea behind the choice of α_0 is to introduce a preference for the collision avoidance direction. Thus, a drone heading to the "north" (for instance, defined by $\operatorname{atan2}(y(\vec{t}), x(\vec{t})) = \frac{\pi}{2}$) will prefer to avoid by going up (because it will start looking in that direction), and a drone heading to the "south" (respectively defined by $\operatorname{atan2}(y(\vec{t}), x(\vec{t})) = -\frac{\pi}{2}$) will prefer to avoid by going down. The use of $z(\vec{t})$ in the computation of α_0 also introduces an additional asymmetry between drones going down or up. This choice of α_0 eases the collision avoidance maneuver.

The application of the algorithm is illustrated by the Fig. 4, showing four iterations of the algorithm.

After the application of the algorithm, the UAV should have found a safe direction. If it is not the case, *i.e*. all of the potential directions are leading to a collision, the drone stops and it leaves the charge of the collision avoidance maneuver to the other UAVs.

C. Velocity Adaptation

The Velocity Obstacle approach offers a safe direction, but does not take into consideration the adaptation of the speed of the UAV.

Intuitively, the adjustment of the speed of the UAV can have the following benefits during a collision avoidance maneuver:

- Reduce the force of the impact of a potential collision by reducing the speed;
- Give more time to perform the collision avoidance maneuver, as the decision will be recomputed based on a sampling time τ; and
- Avoid incoherent behavior and jitter by limiting the velocity gradient.

In order to achieve these objectives, the speed has to be reduced during collision avoidance maneuvers, *i.e*. when the angle Θ between the safe direction and the target direction is not zero. To limit the velocity gradient and to prevent the UAV from going back, the speed v_i is indexed on this angle Θ according to Equation 2, with v_{max} being the maximum value for the UAV speed, and v_{target} the desired speed for the UAV, such that $v_{target} \leq v_{max}$.

$$v_i = \max\left(0, v_{target} \times \left(1 - \frac{\Theta}{\pi}\right)\right) \tag{2}$$

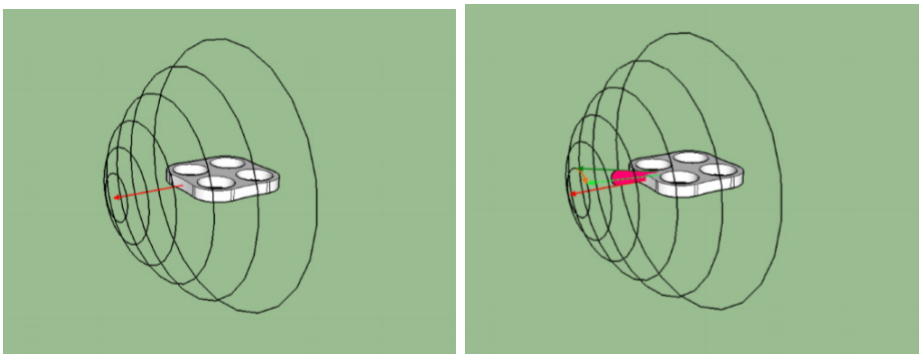
The effective safe target velocity $\vec{s}$ is computed according to Equation 3.

$$\vec{s} = \frac{\vec{s_d}.v_i}{||\vec{s_d}||} \tag{3}$$

III. Simulation and Results

A. Simulation Framework

The used simulation platform is the multi-agent simulation platform for UAVs presented by Lombard et al. [15]. It is based on both AirSim [16] (Fig. 5) which provides a real-time simulation framework for multiple UAVs and the 3D environment, and the SARL [17] multi-agent framework



(a) Target Direction (Inc. 0)

(b) Rotation by θ (Inc. 1)

(c) Rotation by θ (Inc. 2)

(d) Rotation by θ (Inc. 6)

Fig. 4. Iterations of the collision avoidance maneuver algorithm

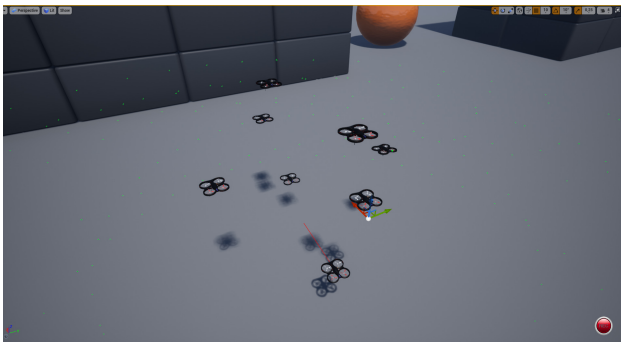


Fig. 5. AirSim simulator with the drones at their initial position

for the implementation of the behavior of the agents. The code allowing developers to run the SARL agents in the AirSim environment is freely available at: https://github.com/alexandrelombard/sarl-airsim-interface [18]. In this simulation, the perceptions of the drones are mostly based on the communication. The other non-communicating obstacles are perceived using proximity sensors (Lidar). Also, there is no delay for the application of the drone's movement.

To test the collision avoidance maneuver, the following simulation scenario is implemented: n drones are placed in a *waiting* state on the surface of a sphere. Then, the drones simultaneously have to cross the sphere to reach the other side. With this configuration, the center of the sphere is a hot point as all UAVs would theoretically pass through this point if there were no other obstacles. This scenario is intentionally not realistic (as having more than 2 drones being in a potential collision is unexpected), in order to create a *worst-case* situation. By provoking potential collisions, it allows to efficiently test the limits of the proposed collision avoidance maneuver.

B. Simulation Results

To evaluate the validity of the proposed solution, the simulations are performed by placing the drones on the sphere and making them simultaneously cross the sphere. In these simulations, the maximal speed was set to $3.5m.s^{-1}$.

Fig. 6 presents the trajectory of two drones simultaneously crossing the sphere. The initial positions of the two drones form a right angle. The idea behind this test is to simulate an intersection of drones at a single point.

Fig. 7 presents the trajectory of four drones simultaneously crossing the sphere. In this scenario, each drone has to manage potential collisions with 3 other drones, one coming from the left, one from the right, and the last one from the front. Fig. 8 presents a similar scenario, with drones' initial positions being distributed in a vertical plane.

In these scenarios, no collisions occurred between the drones, as illustrated by the constant spacing between two successive dots on the figures, showing a relatively constant speed). It highlights the effectiveness of the proposed collision

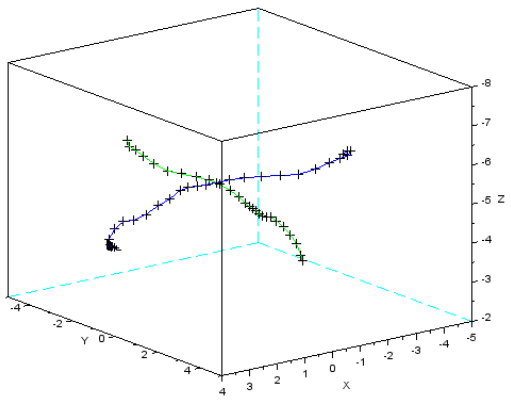


Fig. 6. 3D scatter plot showing the successive positions of two drones with 500ms between successive points

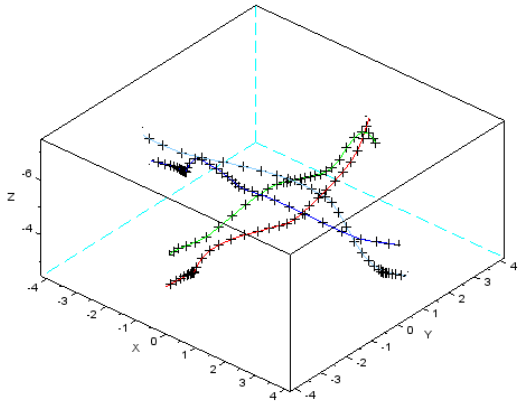


Fig. 7. 3D scatter plot showing the successive positions of 4 drones with 500ms between successive points

avoidance maneuver. It can also be noted that the divergence from the initial direction is also limited.

In Fig. 9 illustrating the successive positions of drones in a simulation involving height UAVs simultaneously crossing the sphere, a collision can be observed (represented by the red and blues lines on the left). It shows the fact that the algorithm cannot ensure a complete safety when in extreme situations. Yet, the other six drones successfully crossed the intersection, adjusting both their trajectories and their speeds.

IV. Conclusion

This paper proposes an adaptation of the VO collision avoidance algorithm, which is traditionally applied to terrestrial robots. The sensors of the drones and their communication capabilities provides a sufficient hardware platform to implement this strategy. Thus, some adaptations of the original approach are proposed: the principle of the CC is extended to 3D, and the collision avoidance maneuver relies on a scan of the safe directions instead of the computation of the reachable avoidance velocities described in [11]. The effective behavior of the approach is then validated using a multi-agent simulator based on both SARL and AirSim frameworks. The simulations let us conclude that the VO approach is really interesting when the number of simultaneous moving obstacles remains low: it can quickly provide a safe solution to avoid a collision, while limiting the impact of the collision avoidance maneuver on the initial trajectory. The behavior is also more controllable than what can be observed when using force-based models. However, as the number of moving obstacles grows, the difficulty to find a safe direction increases, leading to potential collisions. Therefore, the approach can efficiently be included in a more complex strategy as a default solution when the number of moving obstacles is low. Indeed, it is computationally efficient and safe in this situation. While a advanced cooperation-based strategy using communication could be used in the extraordinary case of a lot of simultaneous potential collisions. This interaction-based model needs further research.

Acknowledgments

This work is supported by the Regional Council of Bourgogne Franche-Comté (RBFC, France) within the project UrbanFly 20174-06234/06242.

References

[1] Y. Mualla, A. Najjar, S. Galland, C. Nicolle, I. Haman Tchappi, A.-U.-H. Yasar, and K. Främling, "Between the megalopolis and the deep blue sky: challenges of transport with uavs in future smart cities," in *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 2019, pp. 1649–1653.

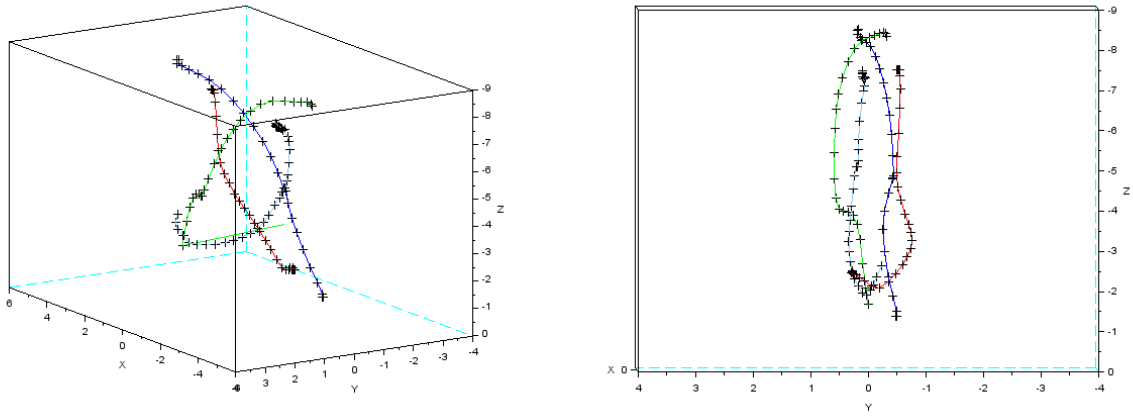
[2] Y. Mualla, A. Najjar, A. Daoud, S. Galland, C. Nicolle, E. Shakshuki *et al.*, "Agent-based simulation of unmanned aerial vehicles in civilian applications: A systematic literature review and research directions," *Future Generation Computer Systems*, vol. 100, pp. 344–364, 2019.

[3] D. Weyns, H. V. D. Parunak, F. Michel, T. Holvoet, and J. Ferber, "Environment for multiagent systems state-of-the-art and research challenges," in *Environments for Multi-Agent Systems (E4MAS)*, S. B. / Heidelberg, Ed., 2005, pp. 1–47.

[4] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The knowledge engineering review*, vol. 10, no. 2, pp. 115–152, 1995.

[5] J.-W. Park, H.-D. Oh, and M.-J. Tahk, "Uav collision avoidance based on geometric approach," in *2008 SICE Annual Conference*. IEEE, 2008, pp. 2122–2126.

[6] V. Cichella, T. Marinho, D. Stipanović, N. Hovakimyan, I. Kaminer, and A. Trujillo, "Collision avoidance based on line-of-sight angle," in *2015 54th IEEE Conference on*



(a) Three quarter view of the positions of the UAVs

(b) Front view of the positions of UAVs

Fig. 8. 3D scatter plot showing the successive positions of 4 drones with 500ms between successive points with their initial positions being at two different levels

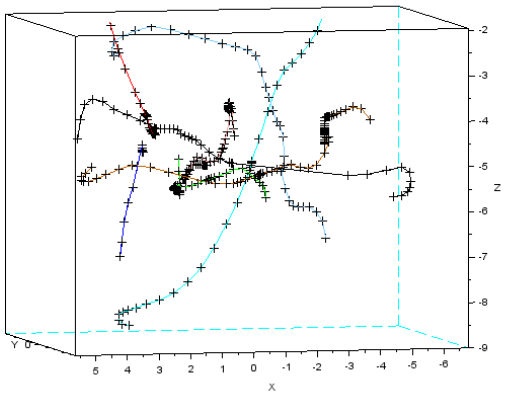


Fig. 9. 3D scatter plot showing the successive positions of 8 drones with 500ms between successive points

Decision and Control (CDC). IEEE, 2015, pp. 6779–6784.

[7] S. S. Ge and Y. J. Cui, "Dynamic motion planning for mobile robots using potential field method," *Autonomous robots*, vol. 13, no. 3, pp. 207–222, 2002.

[8] X. Zheng, S. Galland, X. Tu, Q. Yang, A. Lombard, and N. Gaud, "Obstacle avoidance model for uavs with joint target based on multi-strategies and follow-up vector field," in *The 11th International Conference on Ambient Systems, Networks and Technologies (ANT)*, 2020.

[9] C. W. Reynolds, "Steering behaviors for autonomous characters," in *Game developers conference*, vol. 1999. Citeseer, 1999, pp. 763–782.

[10] D. Helbing and P. Molnar, "Self-organization phenomena in pedestrian crowds," *arXiv preprint cond-mat/9806152*, 1998.

[11] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.

[12] B. Damas and J. Santos-Victor, "Avoiding moving obstacles: the forbidden velocity map," in *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2009, pp. 4393–4398.

[13] Y. I. Jenie, E.-J. Van Kampen, C. C. de Visser, J. Ellerbroek, and J. M. Hoekstra, "Three-dimensional velocity obstacle method for uav deconflicting maneuvers," in *AIAA Guidance, Navigation, and Control Conference*, 2015, p. 0592.

[14] J. A. Douthwaite, S. Zhao, and L. S. Mihaylova, "Velocity obstacle approaches for multi-agent collision avoidance," *Unmanned Systems*, vol. 7, no. 01, pp. 55–64, 2019.

[15] A. Lombard, Y. Mualla, S. Galland, and J. Buisson, "Software architecture for drone simulation in 3d," in *First European Forum for the SARL Users and Developers (EuSarlCon 2019)*, 2019.

[16] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "Airsim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, 2017. [Online]. Available: https://arxiv.org/abs/1705.05065

[17] S. Rodriguez, N. Gaud, and S. Galland, "SARL: a general-purpose agent-oriented programming language," in *the 2014 IEEE/WIC/ACM International Conference on Intelligent Agent Technology*. Warsaw, Poland: IEEE Computer Society Press, 2014.

[18] A. Lombard, "SARL AirSim interface," https://github.com/alexandrelombard/sarl-arisim-interface, 2019.